

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I G.JAAHNAVI(192525277) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Jaahnavi

Assessment Tasks Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.
2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.
3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.
4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.
5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of	Good understanding	Partial understanding	Poor understanding

Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Recommendation Engine for User Behavior

Introduction

A recommendation engine processes user clicks, search queries, and purchases collected from multiple devices. Its main purpose is to understand user behavior and provide personalized recommendations. Since the system handles large and continuously generated data, a scalable big-data architecture is required.

Probable Data Types

The system is likely to handle:

- **Click data:** Pages or products viewed and links clicked.
- **Search data:** Search keywords, filters, and selected results.
- **Purchase data:** Product ID, quantity, price, and transaction details.
- **User data:** User ID, preferences, and profile information.
- **Device data:** Mobile, laptop, tablet, or other device details.
- **Timestamp data:** Time and date of each activity.

Data Ingestion Flow

The probable flow is:

Multiple Devices → Data Collection → Streaming Ingestion → Distributed Storage → Data Processing → Analytics → Recommendation

User activities are collected as events from different devices. A streaming ingestion system can continuously receive these events and transfer them to the storage and processing layers.

Storage

A **distributed data lake or cloud object storage** can store large volumes of historical user activity. A database can store frequently accessed information such as user profiles and recent activities.

Analytics Stages

The analytics process can include:

1. **Data Cleaning** – Removes duplicate or incorrect records.
2. **Feature Extraction** – Identifies useful features such as frequently searched or purchased products.
3. **User Behavior Analysis** – Finds patterns in user activities.
4. **Model Training** – Uses historical data to build recommendation models.

5. Recommendation Generation – Produces relevant products or content for each user.

Techniques such as **collaborative filtering and content-based filtering** can be used.

Design Decisions

A distributed architecture is likely because users generate large amounts of data from multiple devices. Streaming processing helps the system respond to recent user behavior, while historical data supports model training.

Conclusion

The recommendation system combines continuous data ingestion, distributed storage, data processing, and analytics to understand user behavior and provide personalized recommendations efficiently.

2. Ride-Sharing Live Traffic and Demand Forecasting

Introduction

A ride-sharing company displaying live traffic heatmaps and demand forecasts needs to process continuous information from vehicles, users, and traffic sources. The system therefore requires distributed storage and both real-time and historical processing.

Probable Data Sources

The system may collect:

- Vehicle GPS locations
- Ride requests
- Pickup and drop locations
- Vehicle speed

Data Ingestion

The probable architecture is:

Vehicles & Mobile Apps → Data Ingestion → Distributed Message System → Stream Processing → Storage → Analytics

GPS positions and ride requests are continuously transmitted to the backend. A distributed ingestion layer handles the large number of incoming events.

Distributed Storage

The system would probably use:

- **Data lake/object storage:** For historical GPS, trip, and traffic data.
- **Distributed database:** For frequently accessed operational data.
- **Geospatial storage/indexing:** For location and road-related information.

Processing Backbone

Two major processing methods are required.

Real-Time Processing:

Processes current GPS and traffic information to generate live heatmaps.

Batch Processing:

Analyzes historical trips and demand patterns to forecast future ride demand.

Demand Forecasting

The forecasting system can consider:

- Location
- Time of day
- Day of the week
- Previous demand

For example, if a particular location regularly receives a high number of ride requests during a certain period, the system can forecast increased demand there.

Design Decisions

Real-time processing is necessary for live traffic information, while batch processing is useful for detailed historical analysis. Distributed storage and processing allow the system to handle data from a large number of vehicles.

Conclusion

The probable architecture uses distributed ingestion, scalable storage, real-time stream processing, batch analytics, and demand forecasting to provide live traffic information and predict future ride demand.

3. Real-Time Fraud Detection Dashboard

Introduction

A fraud detection dashboard identifies suspicious card transactions in near real time. Because financial transactions are sensitive and time-critical, the system requires secure data processing, streaming analytics, and automatic alerting mechanisms.

Probable Data Sources

The system can collect:

- Transaction amount
- Transaction time
- Card and merchant information
- Transaction location

Streaming Architecture

The probable flow is:

Card Transaction → Secure Ingestion → Stream Processing → Fraud Analysis → Risk Score → Alert Dashboard

Each transaction is treated as an event and processed continuously rather than waiting for batch processing.

Fraud Detection

The system may combine:

- **Rule-based detection:** Identifies known suspicious patterns.
- **Statistical analysis:** Detects unusual transaction behavior.
- **Machine learning:** Learns patterns from previous fraudulent and legitimate transactions.

The system may examine transaction amount, location, frequency, and deviation from normal customer behavior.

Security Mechanisms

Since transaction data is sensitive, important security mechanisms include:

- Data encryption
- Authentication
- Authorization
- Role-based access control

Alerting Mechanism

When a transaction is considered suspicious, the system generates a risk score. If the score crosses a defined threshold, an alert is sent to the fraud dashboard.

Transaction → High Risk → Alert Generation → Dashboard Notification

Storage

Historical transaction information can be stored in distributed storage for analysis and model training, while recent transactions can be maintained in fast-access systems for real-time processing.

Design Decisions

Near-real-time streaming reduces the time required to identify suspicious activity. Combining streaming with machine learning improves detection, while security controls protect sensitive transaction information.

Conclusion

The system combines secure ingestion, real-time stream processing, fraud analytics, distributed storage, and automated alerts to detect suspicious transactions quickly and securely.

4. Social Media Data Management

Introduction

A social media platform handles videos, text comments, hashtags, and user engagement metrics. These represent different forms of big data, including structured, semi-structured, and unstructured data. The system therefore needs different storage and processing methods.

Types of Data

Structured Data

- User IDs
- Post IDs
- Likes and shares
- Timestamps

Semi-Structured Data

- JSON application events
- Hashtag information
- Content metadata

Unstructured Data

- Videos
- Images
- Text comments
- Audio files

Data Flow

The probable architecture is:

Users → Social Media Application → Data Ingestion → Distributed Storage → Processing → Analytics

User activities such as uploading, commenting, liking, and sharing generate data continuously.

Storage Strategy

Different storage systems can be used for different data types:

- **Object storage:** For videos, images, and other large media files.
- **Distributed databases:** For user profiles, posts, and structured information.
- **Data lake:** For storing large collections of raw structured, semi-structured, and unstructured data.

For example, a video can be stored as a large media file while its ID, user ID, timestamp, and hashtags are stored as metadata.

Analytics

The platform can analyze:

- Trending hashtags
- Popular videos
- User engagement

Streaming analytics can process current interactions, while batch analytics can examine historical engagement.

Design Decisions

Using different storage systems is suitable because videos and images require large scalable storage, while structured information requires faster database access. Distributed processing allows analytics to continue even when the data volume becomes very large.

Conclusion

A social media platform would likely combine object storage, distributed databases, data lakes, streaming, and batch analytics. This allows different types of data to be stored and analyzed efficiently at large scale.

5. Manufacturing IoT Platform

Introduction

A manufacturing IoT platform collects machine logs and maintenance events from many plants. It must integrate data from multiple locations, store large volumes of machine information, and analyze the data for monitoring and maintenance purposes.

Probable Data Sources

The system can collect:

- Machine sensor readings
- Machine logs
- Temperature and pressure values
- Machine status
- Timestamps

Integration Flow

The probable architecture is:

Machines/Sensors → IoT Gateway → Data Ingestion → Distributed Messaging → Storage → Processing → Analytics → Maintenance Alert

IoT gateways collect information from machines and send it to the central platform. A distributed messaging system can handle continuous data from many plants.

Storage Strategy

Different storage systems can be used:

- **Data lake/object storage:** For raw machine logs and historical data.

- **Time-series storage:** For continuously changing sensor values.
- **Database:** For equipment details and maintenance records.

Analytics Path

The system can follow:

Raw Data → Data Cleaning → Stream/Batch Processing → Feature Extraction → Analytics → Maintenance Decision

Real-time processing can identify abnormal machine conditions, while batch processing can analyze historical machine behavior.

Predictive Maintenance

Historical data can be used to identify patterns related to equipment failure.

For example:

Normal Machine Data → Abnormal Pattern → Risk Detection → Maintenance Alert

This allows maintenance teams to identify possible problems before major equipment failure occurs.

Data Preparation

Data from different plants may have different formats. Therefore, preprocessing may include:

- Removing duplicate records
- Handling missing values
- Standardizing formats
- Synchronizing timestamps
- Validating sensor readings

Design Decisions

Both streaming and batch processing are useful. Streaming provides immediate machine monitoring, while batch processing supports historical analysis and predictive maintenance. Distributed storage allows the platform to scale as the number of machines and plants increases.

Conclusion

The manufacturing IoT platform would likely use IoT gateways, distributed ingestion, scalable storage, stream and batch processing, and predictive analytics. This architecture supports real-time monitoring, efficient maintenance, and better manufacturing decisions.